Machine Boss: Dynamic Programming Performance Benchmarks

Machine Boss benchmark suite

March 4, 2026

1 Methodology

We benchmark the Forward and Viterbi dynamic programming algorithms across multiple backends in the Machine Boss WFST toolkit:

- **C++ interpreter**: the `boss` CLI generic DP engine (`-L` for Forward, `-V` for Viterbi), using hash-map lookups for transitions
- **C++ compiled**: machine-specific C++ code generated by `boss --cpp64 --codegen`, compiled with `clang++ -O3`; uses direct array access instead of hash-map lookups
- **JAX 1D scan**: Python/JAX using `jax.lax.scan` (sequential)
- **JAX 1D parallel**: Python/JAX using `jax.lax.associative_scan` (parallel prefix)
- **JAX 2D scan**: Python/JAX 2D with sequential outer scan
- **JAX 2D wavefront**: Python/JAX 2D with anti-diagonal wavefront parallelism
- **JAX GPU**: JAX on GPU (skipped if unavailable)
- **JS CPU**: JavaScript CPU fallback via Node.js

1.1 1D problems (generators)

Random generator machines (output-only) with DNA alphabet (A, C, G, T) are tested across:

- Sequence length $L \in \{100, 500, 2000, 10000\}$
- Machine states $S \in \{4, 16, 64\}$

1.2 2D problems (transducers)

Random transducer machines with DNA alphabet are tested across:

- Input length $L_{\text{in}} \in \{10, 50, 200\}$
- Output length $L_{\text{out}} \in \{10, 50, 200\}$
- Machine states $S \in \{4, 16, 64\}$

Each configuration is timed over multiple repetitions after a warmup call (which also triggers JIT compilation for JAX backends). We report mean $\pm$ standard deviation in seconds. Configurations that exceed a timeout threshold are recorded with their probe timing and larger sizes are skipped.

2 Hardware

Property	Value
wideboy.local / Intel(R) Xeon(R) W-3223 CPU @ 3.50GHz	
System	Darwin wideboy.local 24.1.0 Darwin Kernel Version 24.1.0: Thu Oct 10 21:02:27 PDT 2024; root:xnu-11215.41.3~2/RELEASE_X86_64 x86_64
CPU	Intel(R) Xeon(R) W-3223 CPU @ 3.50GHz
Physical cores	8
Logical cores	16
CPU frequency	3.50 GHz
Memory	32 GB
Python	3.14.3
Platform	macOS-15.1.1-x86_64-i386-64bit-Mach-O

3 Results: 1D Forward

Table 1: 1D Forward timings (generators) (seconds, mean ± std).

S	L	`cpp_compiled`	`cpp_interp`
4	100	0.0020 ± 0.0000	0.035 ± 0.000
4	500	0.0020 ± 0.0000	0.034 ± 0.000
4	2000	0.0020 ± 0.0000	0.035 ± 0.000
4	10000	0.0022 ± 0.0000	0.040 ± 0.000
16	100	0.0020 ± 0.0000	0.035 ± 0.000
16	500	0.0020 ± 0.0000	0.036 ± 0.000
16	2000	0.0022 ± 0.0002	0.039 ± 0.001
16	10000	0.0025 ± 0.0001	0.053 ± 0.001
64	100	0.0023 ± 0.0002	0.039 ± 0.000
64	500	0.0021 ± 0.0000	0.041 ± 0.001
64	2000	0.0024 ± 0.0000	0.050 ± 0.000
64	10000	0.0041 ± 0.0001	0.101 ± 0.001

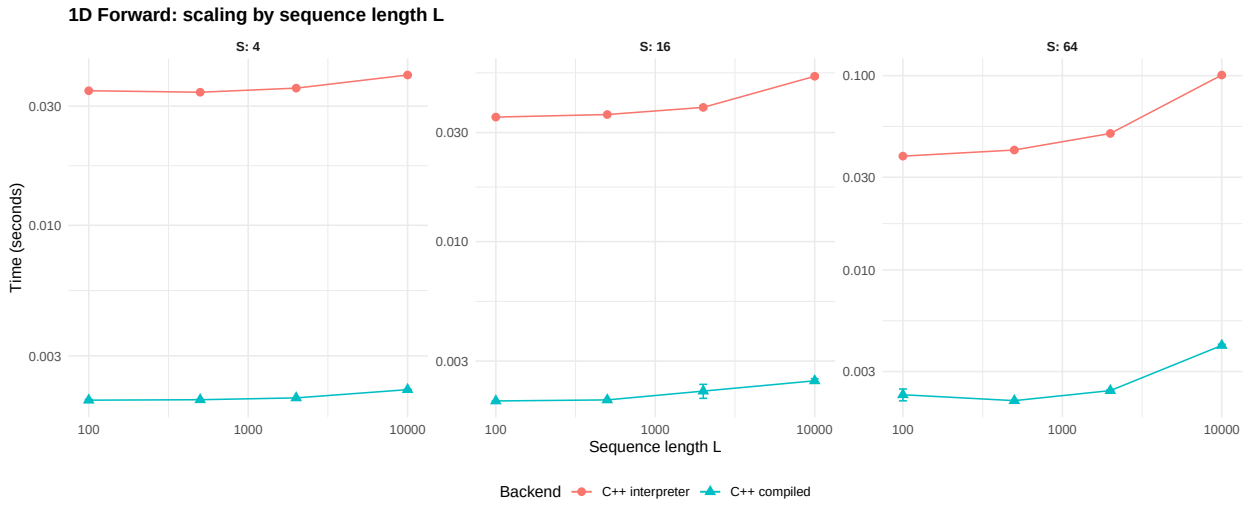

Figure 1: 1D Forward: scaling by sequence length L, faceted by states S. Log-log scale.

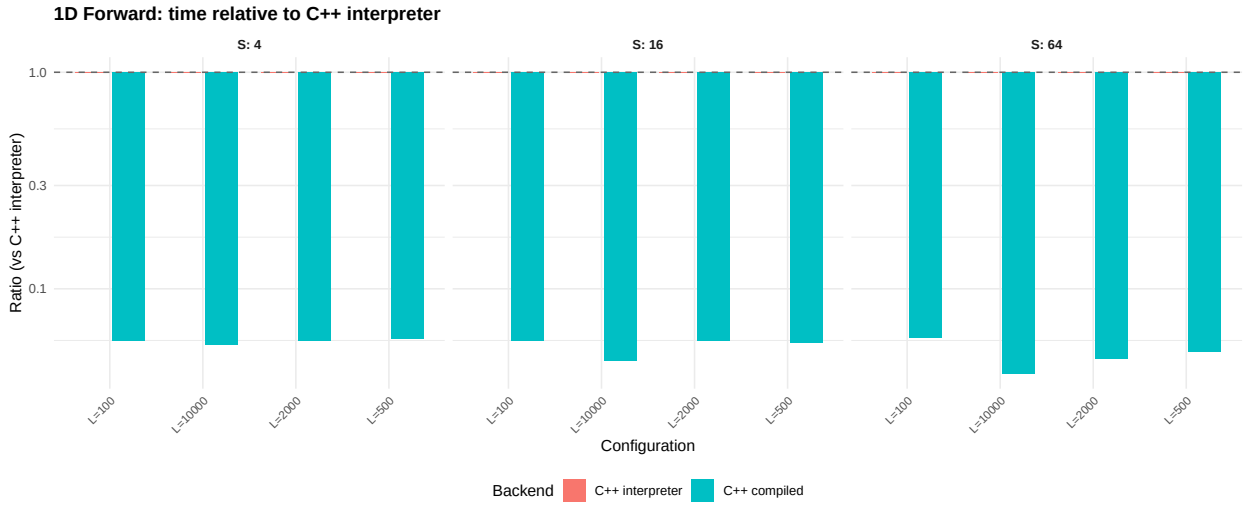

Figure 2: 1D Forward: time relative to C++ interpreter, by configuration.

4 Results: 1D Viterbi

Table 2: 1D Viterbi timings (generators) (seconds, mean ± std).

S	L	cpp_compiled	cpp_interp
4	100	0.0020 ± 0.0000	0.035 ± 0.000
4	500	0.0021 ± 0.0002	0.036 ± 0.001
4	2000	0.0019 ± 0.0001	0.040 ± 0.000
4	10000	0.0022 ± 0.0002	0.064 ± 0.000
16	100	0.0021 ± 0.0001	0.035 ± 0.000
16	500	0.0022 ± 0.0002	0.037 ± 0.000
16	2000	0.0020 ± 0.0000	0.043 ± 0.000
16	10000	0.0022 ± 0.0000	0.075 ± 0.000
64	100	0.0022 ± 0.0002	0.039 ± 0.000
64	500	0.0021 ± 0.0002	0.042 ± 0.001
64	2000	0.0021 ± 0.0000	0.054 ± 0.000
64	10000	0.0026 ± 0.0000	0.125 ± 0.001

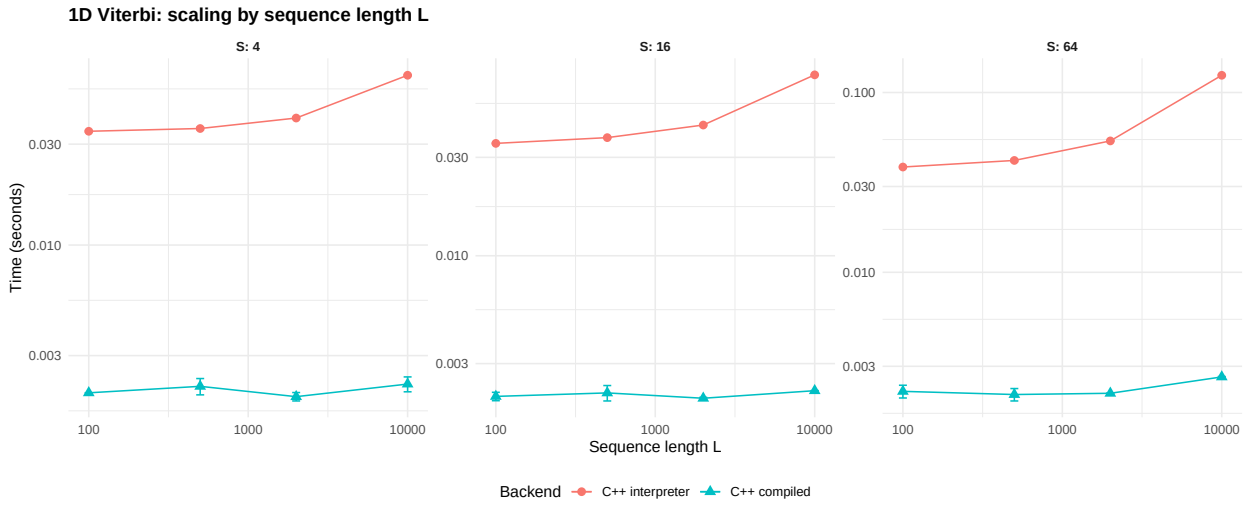

Figure 3: 1D Viterbi: scaling by sequence length L, faceted by states S. Log-log scale.

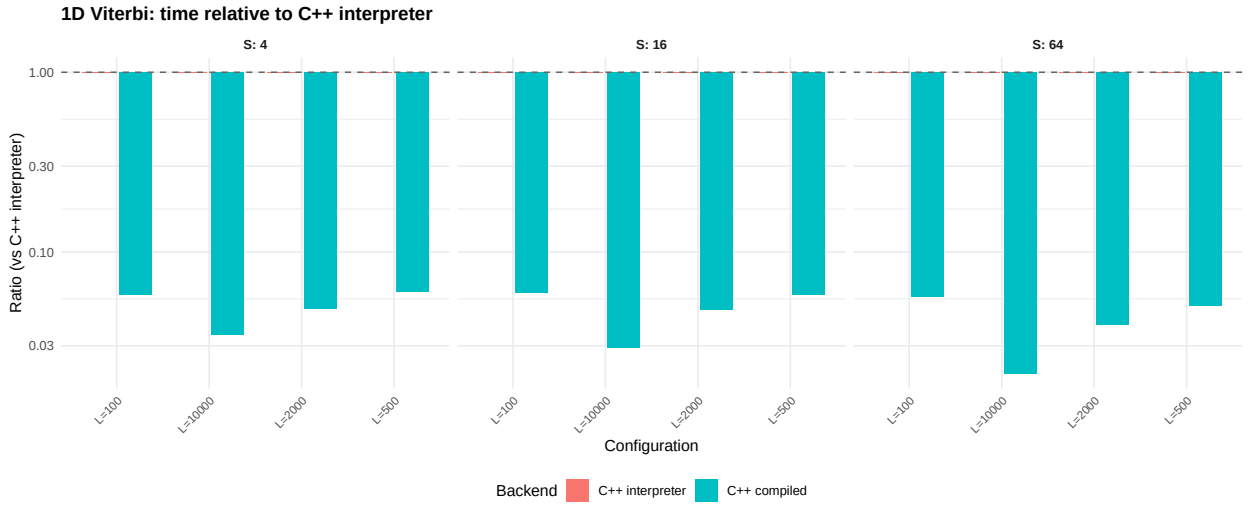

Figure 4: 1D Viterbi: time relative to C++ interpreter, by configuration.

5 Results: 2D Forward

Table 3: 2D Forward timings (transducers) (seconds, mean ± std).

S	L_{in}	L_{out}	`cpp_compiled`	`cpp_interp`
4	10	10	0.0020 ± 0.0000	0.036 ± 0.000
4	10	50	0.0024 ± 0.0002	0.035 ± 0.000
4	10	200	0.0021 ± 0.0000	0.037 ± 0.001
4	50	10	0.0020 ± 0.0000	0.037 ± 0.001
4	50	50	0.0022 ± 0.0002	0.036 ± 0.000
4	50	200	0.0022 ± 0.0002	0.041 ± 0.001
4	200	10	0.0020 ± 0.0000	0.037 ± 0.001
4	200	50	0.0021 ± 0.0000	0.040 ± 0.000
4	200	200	0.0026 ± 0.0000	0.054 ± 0.001
16	10	10	0.0021 ± 0.0002	0.038 ± 0.000
16	10	50	0.0022 ± 0.0002	0.039 ± 0.001
16	10	200	0.0021 ± 0.0000	0.041 ± 0.000
16	50	10	0.0020 ± 0.0000	0.038 ± 0.000
16	50	50	0.0022 ± 0.0000	0.042 ± 0.000
16	50	200	0.0026 ± 0.0000	0.056 ± 0.001
16	200	10	0.0023 ± 0.0002	0.042 ± 0.000
16	200	50	0.0027 ± 0.0002	0.055 ± 0.000
16	200	200	0.0043 ± 0.0000	0.108 ± 0.000
64	10	10	0.0021 ± 0.0000	0.048 ± 0.000
64	10	50	0.0023 ± 0.0002	0.053 ± 0.001
64	10	200	0.0026 ± 0.0000	0.065 ± 0.000
64	50	10	0.0023 ± 0.0002	0.052 ± 0.000
64	50	50	0.0027 ± 0.0000	0.068 ± 0.001
64	50	200	0.0048 ± 0.0000	0.128 ± 0.001
64	200	10	0.0026 ± 0.0000	0.064 ± 0.000
64	200	50	0.0052 ± 0.0002	0.127 ± 0.001
64	200	200	0.013 ± 0.000	0.360 ± 0.001

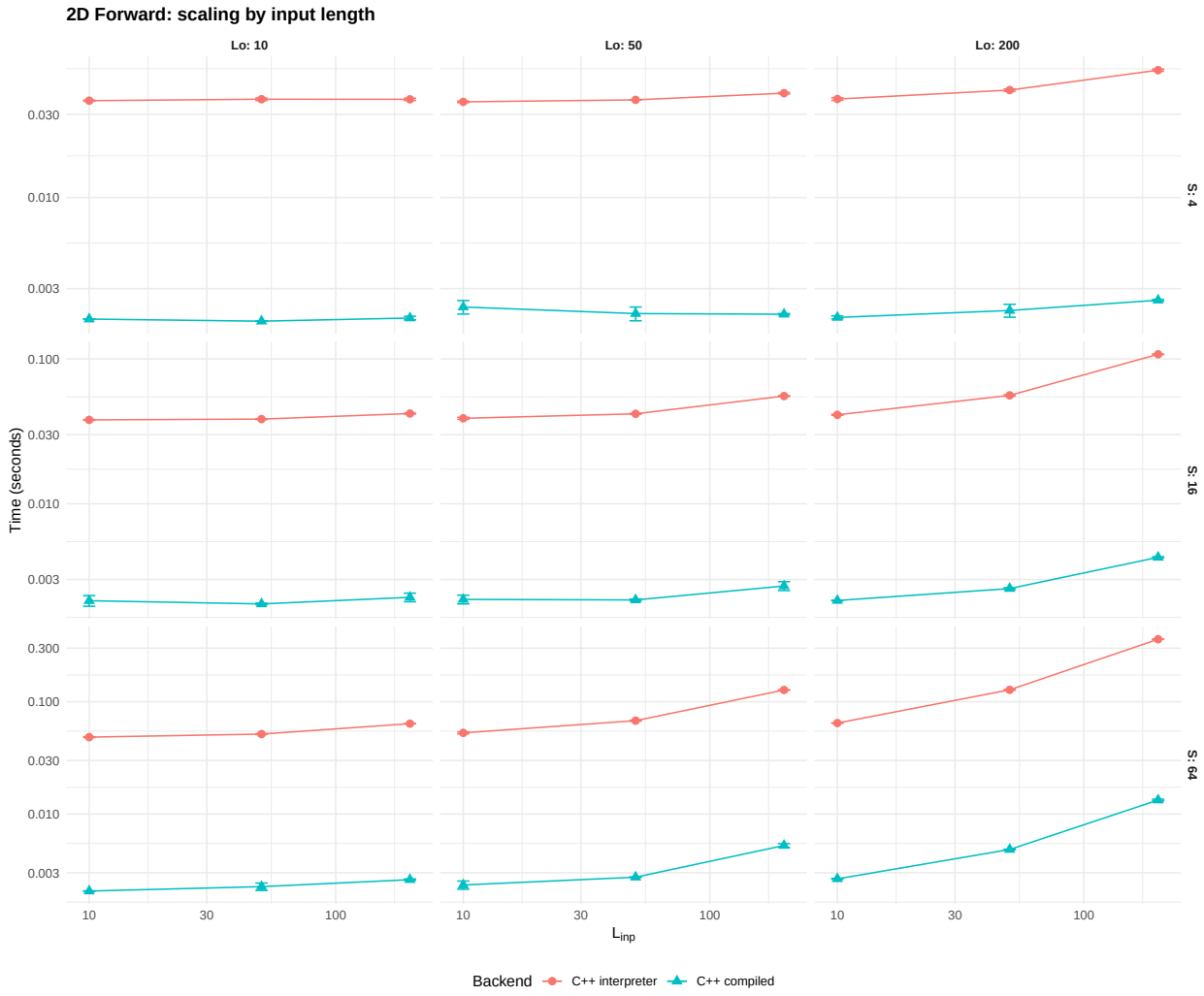

Figure 5: 2D Forward: scaling by input length L_{in}, faceted by states S (rows) and output length L_{out} (columns). Log-log scale.

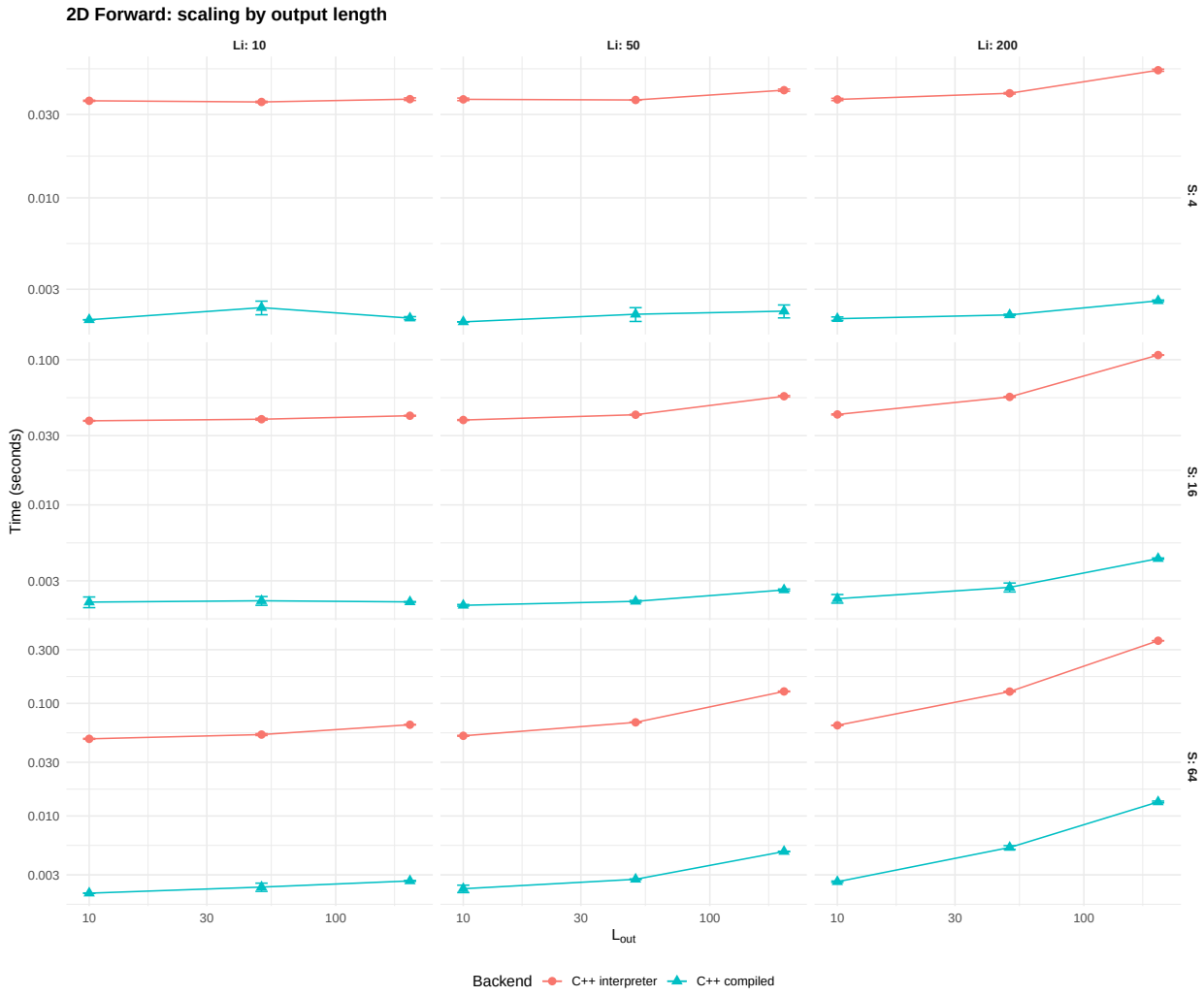

Figure 6: 2D Forward: scaling by output length L_{out}, faceted by states S (rows) and input length L_{in} (columns). Log-log scale.

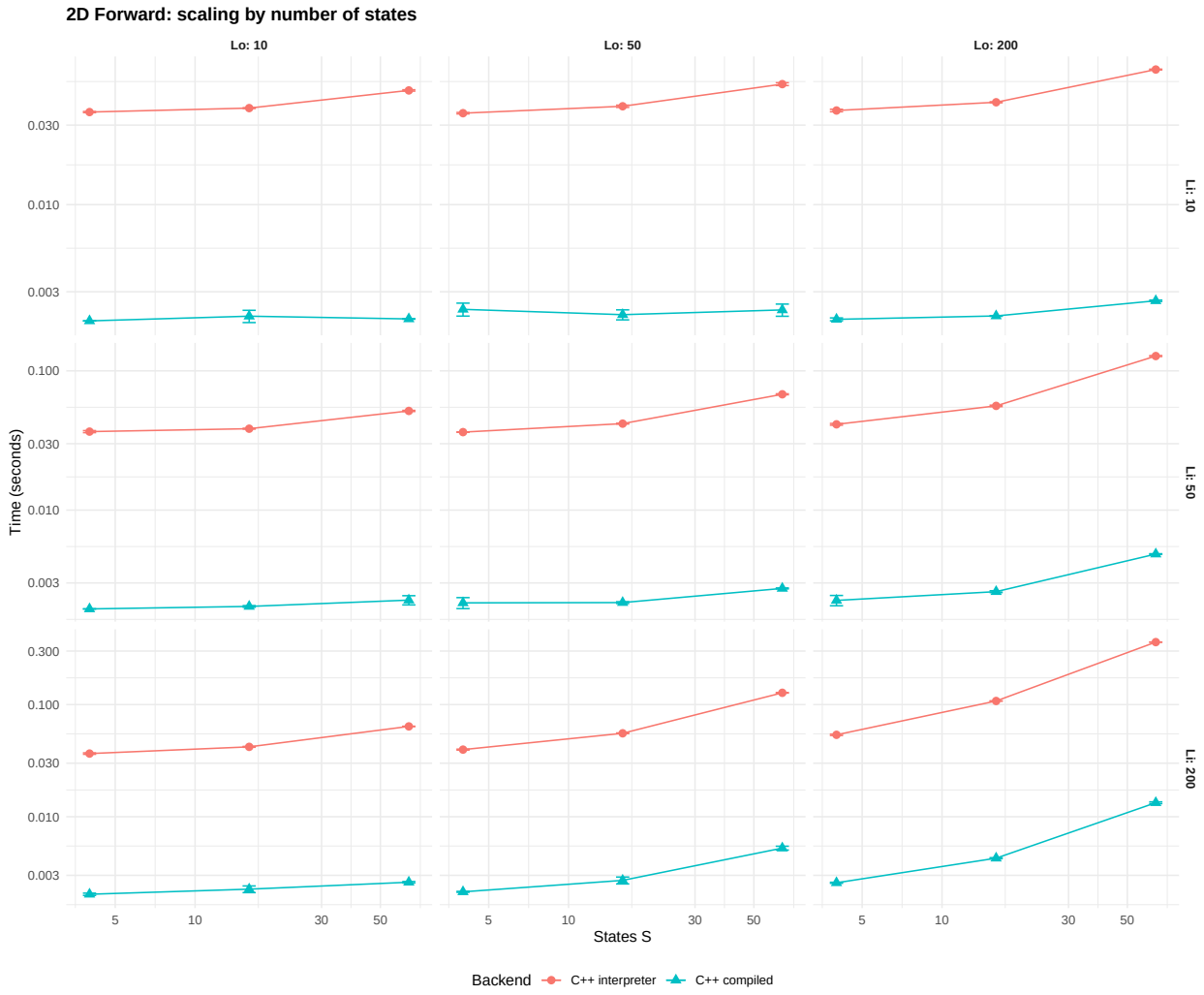

Figure 7: 2D Forward: scaling by states S, faceted by L_{in} (rows) and L_{out} (columns). Log-log scale.

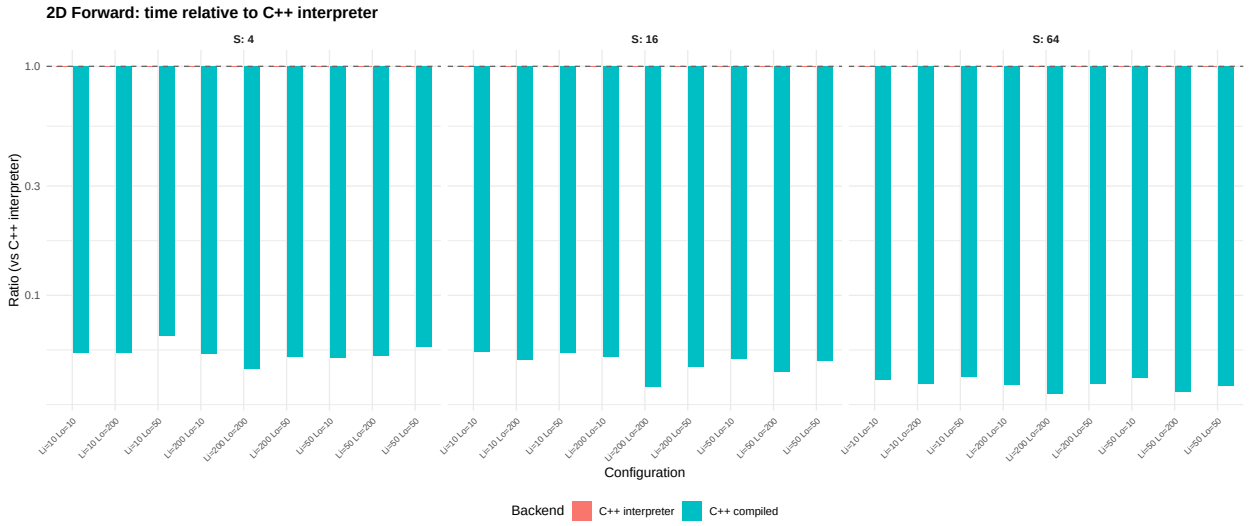

Figure 8: 2D Forward: time relative to C++ interpreter, by configuration.

6 Results: 2D Viterbi

Table 4: 2D Viterbi timings (transducers) (seconds, mean ± std).

S	L_{in}	L_{out}	cpp_compiled	cpp_interp
4	10	10	0.0022 ± 0.0002	0.036 ± 0.000
4	10	50	0.0021 ± 0.0002	0.036 ± 0.000
4	10	200	0.0020 ± 0.0000	0.037 ± 0.000
4	50	10	0.0021 ± 0.0002	0.036 ± 0.001
4	50	50	0.0021 ± 0.0001	0.037 ± 0.000
4	50	200	0.0022 ± 0.0002	0.040 ± 0.000
4	200	10	0.0020 ± 0.0000	0.036 ± 0.000
4	200	50	0.0021 ± 0.0000	0.040 ± 0.001
4	200	200	0.0024 ± 0.0000	0.056 ± 0.002
16	10	10	0.0020 ± 0.0000	0.038 ± 0.001
16	10	50	0.0020 ± 0.0000	0.038 ± 0.000
16	10	200	0.0022 ± 0.0002	0.041 ± 0.000
16	50	10	0.0020 ± 0.0000	0.039 ± 0.000
16	50	50	0.0022 ± 0.0002	0.042 ± 0.001
16	50	200	0.0023 ± 0.0001	0.055 ± 0.001
16	200	10	0.0021 ± 0.0000	0.041 ± 0.000
16	200	50	0.0023 ± 0.0000	0.055 ± 0.001
16	200	200	0.0031 ± 0.0000	0.105 ± 0.000
64	10	10	0.0020 ± 0.0000	0.049 ± 0.000
64	10	50	0.0022 ± 0.0002	0.052 ± 0.000
64	10	200	0.0023 ± 0.0000	0.064 ± 0.001
64	50	10	0.0020 ± 0.0000	0.051 ± 0.000
64	50	50	0.0024 ± 0.0002	0.066 ± 0.000
64	50	200	0.0032 ± 0.0001	0.124 ± 0.000
64	200	10	0.0022 ± 0.0000	0.064 ± 0.000
64	200	50	0.0035 ± 0.0003	0.125 ± 0.001
64	200	200	0.0063 ± 0.0001	0.354 ± 0.003

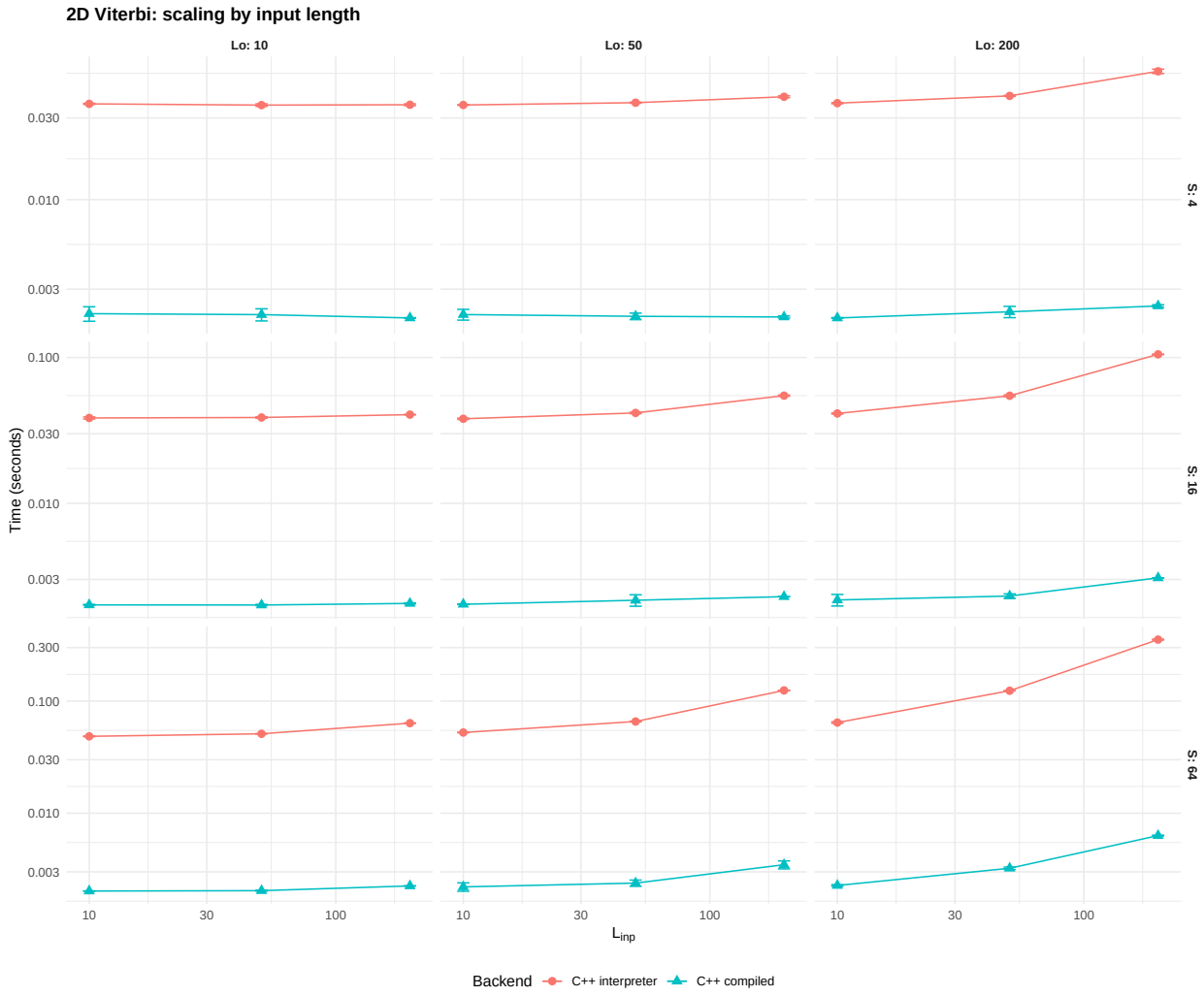

Figure 9: 2D Viterbi: scaling by input length L_{in}, faceted by states S (rows) and output length L_{out} (columns). Log-log scale.

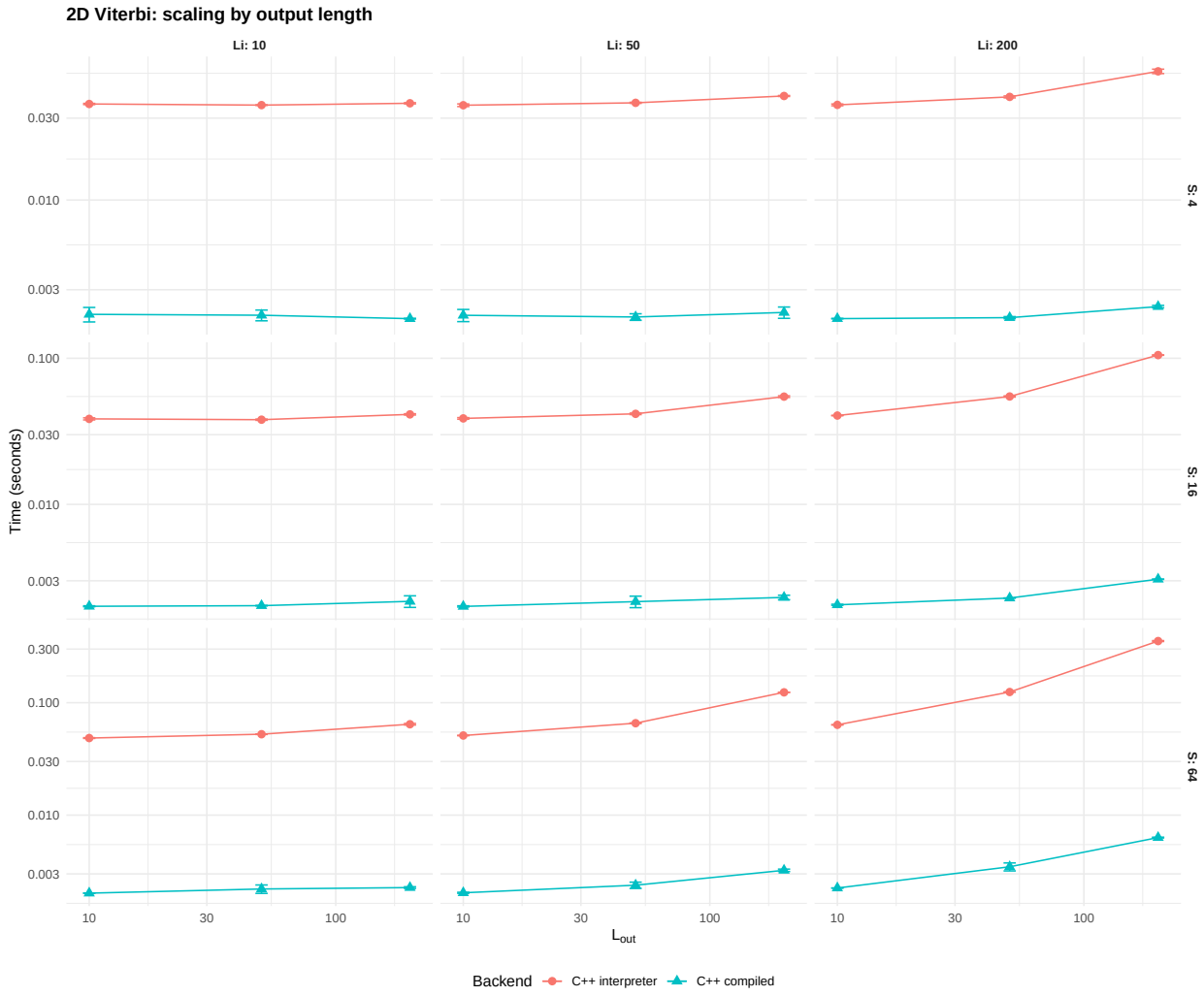

Figure 10: 2D Viterbi: scaling by output length L_{out}, faceted by states S (rows) and input length L_{in} (columns). Log-log scale.

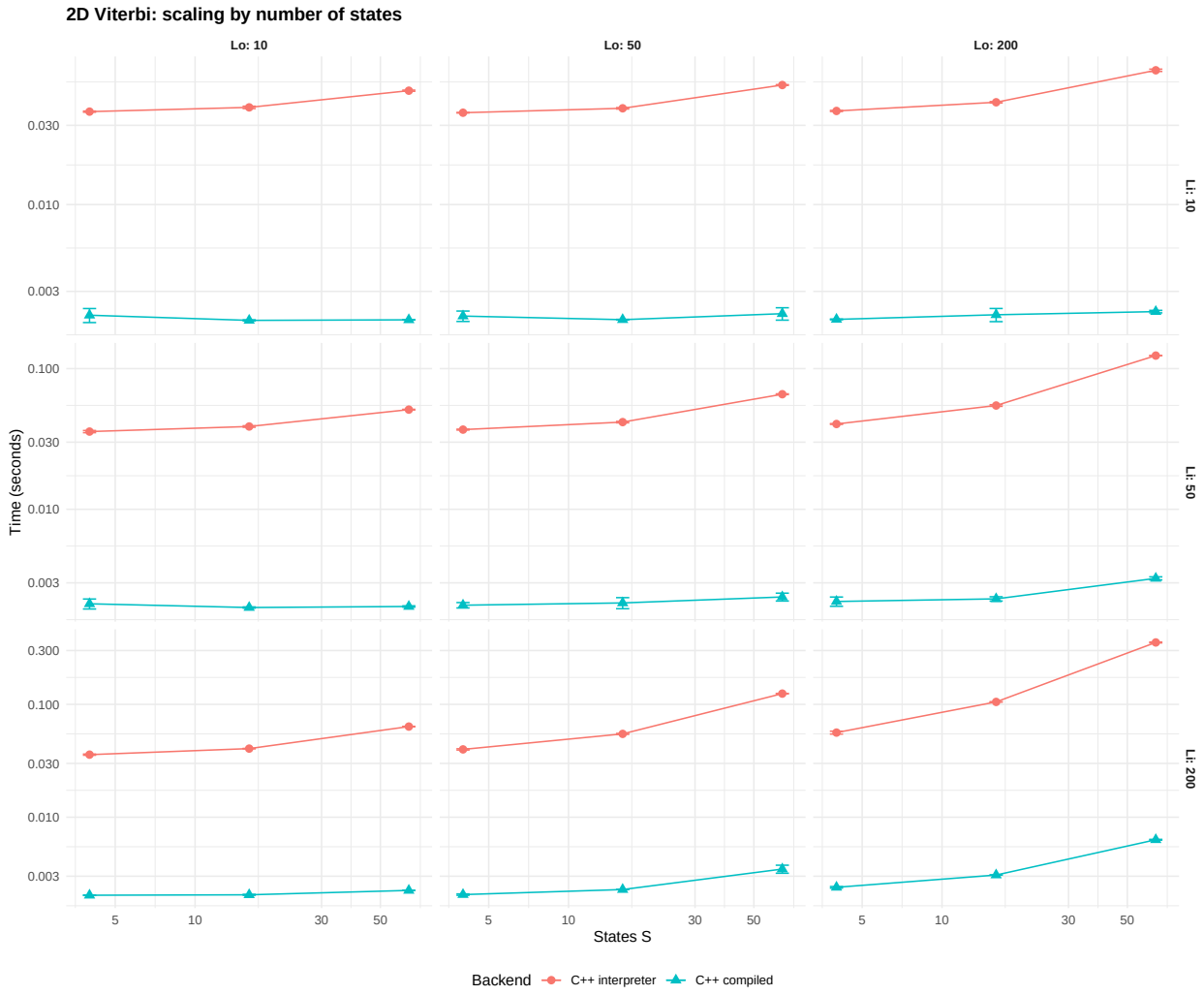

Figure 11: 2D Viterbi: scaling by states S, faceted by L_{in} (rows) and L_{out} (columns). Log-log scale.

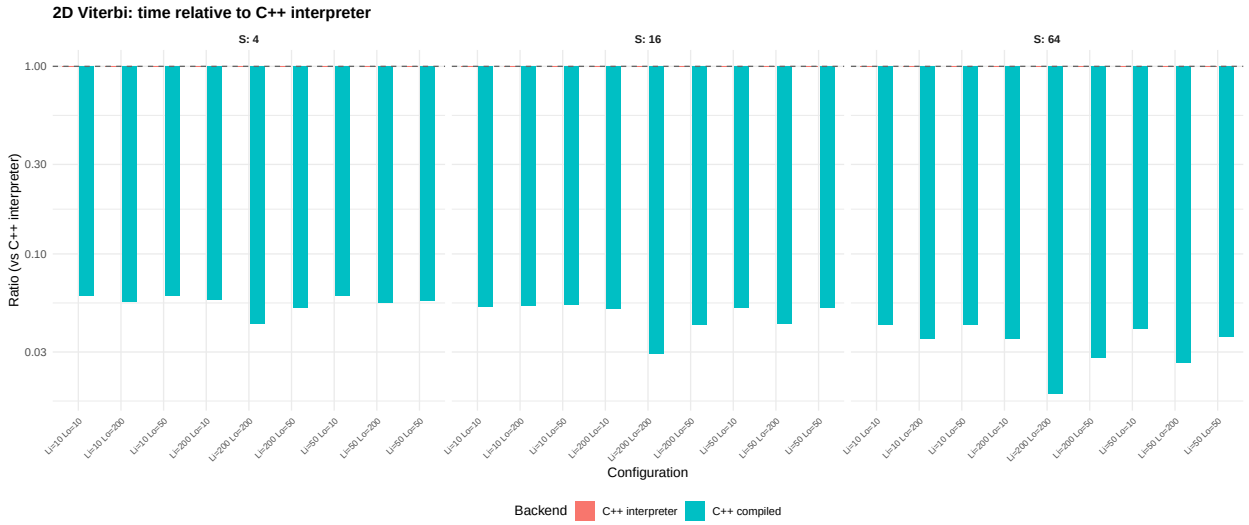

Figure 12: 2D Viterbi: time relative to C++ interpreter, by configuration.